4. Wildcards in Generics — Quick Notes

Why Wildcards?

Normal Inheritance

Student s = new EngineeringStudent();

✅ Valid because:

- EngineeringStudent IS-A Student

Problem with Generics

ArrayList<Student> l1;
ArrayList<EngineeringStudent> l2;

l1 = l2; // ERROR

❌ Compiler Error

Important

Even though: EngineeringStudent → Student

This does NOT mean: ArrayList<EngineeringStudent> → ArrayList<Student>

Wildcard ?

Unknown Type

ArrayList<?> list;

Means:

- Any datatype allowed

Example:

list = new ArrayList<Integer>();
list = new ArrayList<String>();

✅ Both valid

Upper Bounded Wildcard

Syntax

<? extends Student>

Means:

- Student
- OR descendants of Student

Example: ArrayList<? extends Student> list;

Valid:

list = new ArrayList<Student>();
list = new ArrayList<EngineeringStudent>();

❌ Invalid:

list = new ArrayList<Object>();

Lower Bounded Wildcard Syntax

<? super Student>

Means:

- Student
- OR ancestors of Student

Example:

ArrayList<? super Student> list;

Valid:

list = new ArrayList<Student>();
list = new ArrayList<Object>();

❌ Invalid:

list = new ArrayList<EngineeringStudent>();

Wildcards in Functions

General Purpose Function

```
void printStudents(
  ArrayList<? extends Student> list
)
```

Why?

Can accept:

- ArrayList<Student>
- ArrayList<EngineeringStudent>
- ArrayList<ManagementStudent>

Example

ArrayList<EngineeringStudent> eng = new ArrayList<>();

printStudents(eng);

✅ Valid because:

? extends Student

allows descendants.

Collection Wildcard Example

void printCollection(Collection<?> c)

Can accept:

- ArrayList
- HashSet
- TreeSet
- ArrayDeque

with ANY datatype.

Important Point with ?

Datatype unknown.

So:

for(Object x : c)

Use:

- `Object`

NOT:

- `T`
- `Integer`
- `String`

Because actual type unknown.

Why Object Works?

Because:

Object

is root of all classes.

Can call only Object methods:

- toString()
- equals()
- hashCode()

Wildcards in Java Docs

Example:

boolean addAll(Collection<? extends E> c)

Meaning

Can add:

- E type
- OR descendants of E

Example: List<Student>

can add: List<EngineeringStudent>

Important Quick Points

Wildcard	Meaning
`<?>`	Unknown type
`<? extends T>`	T or child classes
`<? super T>`	T or parent classes

Easy Memory Trick

Keyword	Direction
extends	Downward (children)
super	Upward (parents)